universität freiburg

Intelligent Embedded Systems Lab

Hardware-Praktikum 2026

O. Amft und L. Häusler, Intelligent Embedded Systems Lab,
Institute for Computer Science, Albert-Ludwigs-University Freiburg, Freiburg im Breisgau

Exercise sheet 3 (26 points)

Goal:

Back to higher-level programming! The objective of this exercise is to develop embedded sensing and monitoring applications using the Seeed XIAO platform. You will acquire data from multiple environmental sensors, apply calibration and data processing techniques, implement non-blocking real-time operation, handle sensor faults robustly, and design a smart monitoring system that integrates local visualization, state-based decision making, and wireless BLE communication.

Task 1 (4 points): Environmental Light Categorization

In this task, you will read ambient light intensity using a Grove Light Sensor and classify it into discrete categories. Connect the sensor to the analog port of the expansion board using the included cable. To improve measurement precision, configure the ADC to 12-bit resolution. Due to sensor non-linearity, apply empirical calibration using the following values:
Minimum (dark): 50
Maximum (bright): 3500

i) Sensor Readout and ADC Configuration: Read the analog value from the light sensor and configure the ADC to 12-bit resolution. (1)

ii) Calibration and Normalization: Clamp the raw sensor value to the calibrated range [50, 3500], and map it to a normalized (integer) scale of 0–100. (1.5)

iii) Categorization: Classify the normalized value into LOW (< 30), MEDIUM (30–69), and HIGH ($\geqslant 70$). (0.5)

iv) Non-blocking Execution and Serial Output: Implement periodic measurement using a non-blocking approach (no *delay()*), and output raw value, normalized value, and category every 500 ms via the Serial monitor. (1)

Notes:

- Use named constants for all timing intervals, thresholds, and calibration values to ensure code readability and maintainability.
- Select the Seeed nRF52 mbed-enabled Board in the Arduino IDE. This will appear as `XIAO nRF52840 Sense (No Updates)`. If you want to use the onboard Seeed nRF52 Boards library, you must include `#include <Adafruit_TinyUSB.h>` at the top of your source code like in previous exercises.
- Use a baud rate of `115200`. You may need to open the Serial Monitor before the program can compile and upload successfully.

Task 2 (4 points): Robust Temperature and Humidity Monitoring

In this task, you will implement a robust environmental monitoring system using a DHT11 temperature and humidity sensor. The system must periodically read temperature (°C) and relative humidity (%) values and handle potential sensor failures gracefully. The DHT11 sensor uses a single digital interface which can be addressed using pin D7 on the extension board. Required Libraries:

- `<DHT.h>`
- `<math.h>`

i) Sensor Initialization and Periodic Readout: Initialize the DHT11 sensor and read temperature and humidity values every 2 seconds using a non-blocking approach (no *delay()*). (1)

ii) Error Handling and Fault Tolerance: Detect invalid readings (NaN values). Maintain a failure counter, reset it to zero immediately upon a successful sensor read ("Reset on Success"), and reuse the last valid readings if a measurement fails. Ensure the system continues operating even after failed readings. (1.5)

iii) Dew Point Calculation: Compute the dew point temperature based on the measured temperature and relative humidity using the Magnus formula ($a = 17.62$, $b = 243.12\,^{\circ}$C). Ensure you use the natural logarithm for the calculation. (1)

iv) Serial Output and Warning System: Print temperature, humidity, and dew point values via the Serial monitor. Display a specific warning message (e.g., `[WARN] DHT failures: N`) only if the consecutive failure counter exceeds a threshold of 5. (0.5)

Example Output:

```
T=24.2C  RH=45.0%  dewPoint=11.3C
T=24.3C  RH=44.8%  dewPoint=11.2C
[WARN] DHT failures: 6
T=24.3C  RH=44.8%  dewPoint=11.2C  (Using last valid)
```

Note: Print all floating-point values with one decimal place.

Task 3 (4 points): Air Quality Monitoring with SGP30

In this task, you will implement an air quality monitoring system using the SGP30 sensor via the I2C interface. In comparison to the first exercise, we will use the provided library to make reading sensor data more straightforward. The sensor provides: equivalent CO_2 (eCO_2) in ppm, and Total Volatile Organic Compounds (TVOC) in ppb.

Note: The sensor requires a warm-up period of 15 seconds before delivering stable readings.
Required Libraries:

- `<Adafruit_SGP30.h>`

i) Sensor Initialization and I2C Setup: Connect and initialize the SGP30 sensor and start air quality measurements. (1)

ii) Periodic Measurement: Read eCO_2 and TVOC values at a rate of 1 Hz using a non-blocking approach (no *delay()*). (1)

iii) Warm-up Handling: Implement a warm-up phase (15 seconds): indicate that the sensor is warming up. During warm-up, continue measuring/printing at 1 Hz but mark readings as `UNSTABLE/WARMUP`. Switch to "ready" state after the warm-up period. (1)

iv) Error Handling and Serial Output: Detect and report measurement failures (call `IAQmeasure()` once per second; if it returns false, treat as measurement failure). On failure, print a clear error line; on success, print values. Output eCO_2 and TVOC values in a correctly formatted way and ensure correct handling of integer data types. (1)

Task 4 (14 points): Smart Plant Monitoring Station

In this task, you will design and implement a smart plant monitoring system integrating multiple sensors, local visualization, and wireless communication. Your system must continuously acquire environmental data, evaluate plant health, and provide both local and remote feedback.
Required Libraries:

- `<ArduinoBLE.h>`
- `<Adafruit_SGP30.h>`
- `<U8g2lib.h>`
- `<DHT.h>`
- `<Wire.h>`

i) Asynchronous Sensor Data Acquisition: Read data from all sensors using non-blocking timing (no *delay()*): light sensor every 500 ms; temperature & humidity every 2 s; air quality (SGP30 eCO_2) every 1 s. (3)

ii) Intelligent FSM and Health Evaluation: Design a decision-making engine using a Finite State Machine (FSM) with an enum structure. (1)

iii) Warm-up, Scoring, and State Transitions: (5)

- Warm-up Phase (INIT): Upon power-up, remain in `STATE_INIT` for 30 seconds. During INIT, display `"Warming Up..."` and do not calculate health scores / do not transition to other health states.
- Health Scoring Logic: Once operational, compute a cumulative Health Score (0–100) every 1 second using the latest available sensor values (since DHT is sampled every 2 seconds, temperature/humidity values may be reused for intermediate score updates). Each condition contributes +25 points if within range: light 25–90 % (use Exercise 3.1 normalization), temperature 18–30 °C, humidity 30–75 %, $eCO_2 < 1200$ ppm.
- State Transitions: Score $\geqslant 75 \rightarrow$ `STATE_HEALTHY`; Score 50–74 $\rightarrow$ `STATE_ATTENTION`; Score $< 50 \rightarrow$ `STATE_STRESSED`.
- Critical Priority Override: Regardless of score, if $eCO_2 > 2200$ ppm, transition immediately to `STATE_STRESSED`.

iv) Local Visualization (OLED): Display current system state, temperature (°C), humidity (%), light level (%), and eCO_2 (ppm). Update the display at a stable rate of $\sim$2 Hz. Avoid unnecessary redraws. (2)

v) Alert System (LED & Buzzer): Implement state-dependent feedback (non-blocking): HEALTHY → LED ON (steady); ATTENTION → LED toggle every 500 ms; STRESSED → LED toggle every 250 ms. Additionally: if $eCO_2 > 2200$ ppm, activate a buzzer alarm of your choice. (2)

vi) BLE Telemetry: Transmit telemetry via BLE at 1 Hz (temperature, humidity, light level (%), eCO_2 (ppm), system state). Use a single formatted UTF-8 string (e.g., `snprintf`). Example: `T=24.5 H=45 L=80 C=1200 S=HEALTHY`. Notify requirement: the data must be exposed via a BLE characteristic that supports Notifications (Notify), so nRF Connect can subscribe. (1)

Testing & Validation Requirement (nRF Connect for Mobile): To verify the BLE telemetry functionality, install the *nRF Connect for Mobile* application on your smartphone:

- Android: https://play.google.com/store/apps/details?id=no.nordicsemi.android.mcp
- iOS: https://apps.apple.com/app/nrf-connect-for-mobile/id1054362403

Testing procedure:

1. Upload and start your program.
2. Open the nRF Connect app and scan for nearby BLE devices.
3. Connect to your device.
4. Locate the telemetry characteristic and enable Notifications (triple-arrow icon).

5. Set the displayed data format to *UTF-8 / Text*.
6. Verify that the transmitted status string is displayed correctly, e.g.:

```
T=24.5 H=45 L=80 C=1200 S=HEALTHY
```

The BLE implementation is considered valid only if the complete telemetry string can be received and displayed as readable text in nRF Connect.

Submission:

Archive your programs in a ZIP file and upload it to the exercise portal. Check the archive file for completeness. Your submission should contain 4 programs: Task 1, Task 2, Task 3, and Task 4. Task parts that have been modified or taken out by other tasks should be commented out.